

ServiceNow Training Lecture Notes

Client-Server Architecture & GlideUser API

These notes summarize the concepts, code blocks, and execution steps explained in the training session regarding ServiceNow client-server boundaries and client-side user queries.

1. ServiceNow Client vs. Server Architecture

ServiceNow divides its script configurations and database execution into two distinct environments:

- Client Side (User Browser):

This is where forms take inputs. Scripts running here manipulate form fields, hide/show tables, and perform UI validation. Runs JavaScript + client-side APIs.

- Server Side (Cloud Database):

This is where values are saved. Once a form is submitted, data is written into database records. Runs JavaScript + server-side APIs.

Key Boundary Rule:

Client-side APIs (like `g_user` or `g_form`) cannot be used in server-side scripts (they fail or throw runtime errors). Conversely, server-side APIs (like `gs` or `GlideRecord`) cannot be used in client-side scripts.

2. Important Client-Side APIs

ServiceNow provides specific APIs targeted for forms and list layouts. These include:

- g_user (GlideUser): Fetches information about the currently logged-in user profile.
- g_form (GlideForm): Configures field parameters dynamically.
- GlideAjax: Triggers asynchronous server calls to Script Includes.
- g_navigation / g_list / GlideModal: Manages browser overlays, lists, and windows.

3. GlideUser Client-Side API (g_user)

The g_user object is pre-instantiated inside browser scripts, providing details on user profiles.

NOTE: To test client scripts on a live form, open a record page (like Incident) and press Ctrl + Shift + J. This opens ServiceNow's built-in JavaScript Executor. You can write and run client-side scripts there instantly.

A. Properties vs. Methods

- Properties (Fields): Static variables describing user fields. Do not end with parenthesis ().
- Methods (Functions): Executable commands that return values. Must end with parenthesis ().

Core properties:

- g_user.userID: Sys ID of the user record.
- g_user.userName: Login username (e.g. 'admin').
- g_user.firstName: User's first name.
- g_user.lastName: User's last name.

Core method:

- g_user.getFullName(): Combines first name and last name into a single display string.

```
// Read properties and call the getFullName method
var fName = g_user.firstName;
var lName = g_user.lastName;
var fullName = g_user.getFullName();
var loginName = g_user.userName;
var sysId = g_user.userID;

// Output alerts
alert('User Full Name: ' + fullName);
alert('User Sys ID: ' + sysId);
```

B. User Role Checking Methods

You can verify the roles assigned to the currently logged-in user dynamically:

1. hasRole(roleName)

Returns true if the user possesses the role. Admins automatically return true for this check (they bypass standard checks).

```
if (g_user.hasRole('itil')) {  
    alert('User has itil role or is an administrator.');
```

2. hasRoleExactly(roleName)

Returns true ONLY if the user has that specific role explicitly assigned. It does NOT automatically return true for Admins.

```
if (g_user.hasRoleExactly('security_admin')) {  
    alert('User has the security_admin role assigned explicitly.');
```

3. hasRoles()

Returns true if the user has any roles assigned to them (excluding the default 'public' role).

4. hasRoleFromList(rolesList)

Returns true if the user has at least one of the roles listed in the comma-separated string list. Just like hasRole(), it will automatically evaluate to true for administrators.

```
var hasAccess = g_user.hasRoleFromList('itil, catalog_admin, asset');  
if (hasAccess) {  
    alert('User is authorized to fulfill tickets.');
```

4. Key Takeaways for Developers

- Client Debugging: alert() is useful for developer debug tracing but halts user page execution. Replace it with g_form.addInfoMessage() in final production configurations.
- Impersonation: When you impersonate another user profile (e.g. impersonating a non-admin user), g_user properties update to reflect that user's profile and active roles list.
- JavaScript Executor: Ctrl + Shift + J is a powerful admin debugging tool to run ad-hoc scripts directly on any ServiceNow UI form.